| | |
|---|---|
| | **Carátula para entrega de prácticas** |
| Facultad de Ingeniería | Laboratorios de docencia |

# Laboratorio de Computación Salas A y B

*Profesor(a):* René Adrián Dávila Pérez

*Asignatura:* Programación Orientada a Objetos

*Grupo:* 07

*No de Prácticas(s):* 9

*Integrante(s):* 323078401

323115296

323082392

323139441

323253916

*No. de lista o brigada:* 04

*Semestre:* 2027-I

*Fecha de entrega:* 30/10/2026

*Observaciones:*

CALIFICACIÓN: _____

# Índice

**1. Introducción** **3**

1.1. Planteamiento del problema . . . . . 3

1.2. Motivación . . . . . 3

1.3. Objetivos . . . . . 3

**2. Marco Teórico** **3**

**3. Desarrollo** **5**

3.1. Prácticas . . . . . 5

3.2. Proyecto . . . . . 8

**4. Resultados** **10**

4.1. Resultados de la Práctica 9 . . . . . 10

**5. Conclusiones** **17**

**6. Referencias** **17**

# 1. Introducción

## 1.1 Planteamiento del problema

El problema de esta práctica consiste en la representación de las prácticas 5, 7, 8 y la etapa 2 del proyecto en la elaboración de diagramas de clases con cardinalidad y los diagramas de secuencia. Se busca conseguir que las multiplicidades asignadas, los tipos de relación y las trazas de ejecución modeladas encuentren un sustento verificable en las instrucciones, referencias y métodos del código fuente.

## 1.2 Motivación

Aprender a representar código en Lenguaje de Modelado Unificado que nos proporciona un estándar visual para comunicar el diseño del software con mayor claridad. Aparte, aprender a modelar requiere no solo elaborar diagramas que luzcan coherentes a nivel gráfico, sino también se debe poder justificar multiplicidad y mensaje frente a la realidad del programa ejecutable.

## 1.3 Objetivos

- Elaborar los diagramas de clases con cardinalidad y los diagramas de secuencia correspondientes a las Prácticas 5, 7, 8 y a la Etapa 2 del Proyecto, verificando la consistencia entre la estructura estática y el comportamiento dinámico.
- Comprender la semántica de los elementos presentes en cada diagrama, identificando participantes, multiplicidades, tipos de relaciones, líneas de vida, activaciones y flujos de mensajes.

# 2. Marco Teórico

El Lenguaje de Modelado Unificado (**UML**) estandariza los esquemas gráficos requeridos para describir estos aspectos sin ambigüedades técnicas [2].

El **diagrama de clases** modela la forma interna del software mediante clasificadores discretos que representan abstracciones computacionales [2]. Una clase se representa mediante un clasificador rectangular dividido convencionalmente en compartimentos que segregan su nombre, sus atributos y sus operaciones o métodos [1]. El modelado exige documentar la visibilidad de cada miembro mediante modificadores

estandarizados, distinguiendo elementos públicos (+), privados (-) y protegidos (#), lo que traduce a nivel visual el concepto de encapsulamiento [3].

Las conexiones entre clasificadores expresan la naturaleza de su interacción en memoria [1]. La relación de **dependencia**, indicada mediante una flecha discontinua abierta, denota una vinculación débil o temporal en la cual una clase utiliza los "servicios" de otra clase dentro del alcance de una operación local o parámetro de método [2]. Por su parte, la **asociación simple** representa una conexión estructural perdurable respaldada comúnmente por variables de instancia que preservan referencias hacia otros objetos durante su ciclo de vida [3].

La **cardinalidad** o **multiplicidad** constituye la especificación que cuantifica las instancias que pueden o deben asociarse válidamente entre dos clasificadores [1]. La notación UML representa la multiplicidad mediante rangos continuos o valores discretos en los extremos de la relación, donde una cifra como 1 representa una presencia estricta y obligatoria, un intervalo acotado como 0..1 denota opcionalidad, y rangos como 0..* o 1..* delimitan cardinalidades mínimas y máximas abiertas [2].

El **diagrama de secuencia** captura la colaboración ordenada en el tiempo de un conjunto de objetos para satisfacer un escenario o caso de uso determinado [1]. Los participantes se disponen horizontalmente en la parte superior del diagrama bajo la convención identificador : Tipo, y de cada uno desciende una línea vertical discontinua denominada **línea de vida**, la cual traza la existencia temporal del objeto durante la ejecución del proceso [2]. A lo largo de la línea de vida se superponen rectángulos alargados conocidos como **focos de activación**, que delimitan el periodo exacto en el cual el objeto mantiene el control de procesamiento o se encuentra aguardando el resultado de una invocación delegada [3].

Las transiciones entre participantes se formalizan mediante mensajes ordenados de arriba hacia abajo sobre el eje temporal [1]. La creación de **instancias** se denota mediante flechas dirigidas explícitamente a la cabeza del clasificador receptor, comúnmente acompañadas de «constructor» [3]. La auto-llamada ilustra una invocación que el objeto realiza sobre un método perteneciente a su propia clase, modelándose como un lazo que retorna a la misma barra de activación [1].

Los retornos de control o resultados derivados de métodos no vacíos se indican mediante flechas discontinuas abiertas orientadas hacia el emisor original [2]. Finalmente, el modelado del comportamiento estructurado en secuencias complejas requiere el empleo de fragmentos combinados delimitados por marcos rectangulares con un operador formal en su esquina superior izquierda [1]. Destacan el operador **loop**, que encapsula la repetición de interacciones sujeta a una condición, y los operadores condicionales **alt** u

**opt**, para representar bifurcaciones y despachos polimórficos de métodos virtuales en tiempo de ejecución [1], [3].

# 3. Desarrollo

## 3.1 Prácticas

El programa desarrollado en la **Práctica 5** fue diseñado para, por un lado, representar una entidad laboral encargada de registrar su información, validar que las percepciones económicas no asuman magnitudes negativas y calcular incrementos porcentuales; por otro lado, un registro cronológico encargado de administrar componentes de fecha en el calendario y admitir modificaciones individuales mediante métodos específicos. El sistema se estructura a través de dos clases de dominio, **Empleado** y **Fecha**, complementadas por dos controladores de prueba con método principal, **MainEmpleado** y **MainFecha**.

El diagrama de cardinalidad modela dos subsistemas aislados sin dependencias mutuas ni jerarquías de generalización. El primer subsistema vincula al controlador **MainEmpleado** con la clase de dominio **Empleado**, mientras que el segundo asocia a **MainFecha** con **Fecha**. Ambos emplean relaciones de dependencia e instanciación de uso temporal, trazadas con flechas discontinuas abiertas.

La relación entre **MainEmpleado** y **Empleado** establece una multiplicidad de 1 en el extremo del controlador y de 2 en el extremo de la entidad de dominio, complementada con el rol «instancia y compara». Esta cardinalidad fija refleja la restricción del algoritmo implementado, el cual instancia exactamente dos referencias de empleados para contrastar sus percepciones salariales antes y después de un incremento. Por su parte, la dependencia entre **MainFecha** y **Fecha** declara una multiplicidad de 1 a 1 acompañada del rol «instancia y muta», expresando que el flujo del programa "cliente" centra su ciclo de vida en inicializar y modificar una única instancia temporal. Las clases exhiben con precisión sus compartimentos de atributos privados y métodos públicos, destacando la presencia de las operaciones **validarSalario**, **salarioAnual** y **aumentarSalario** en **Empleado**, así como los mutadores de día, mes y año en **Fecha**.

El diagrama de secuencia describe el ciclo de vida completo de ejecución en dos fases consecutivas. La interacción inicia con un actor externo denominado **Usuario** que detona la ejecución enviando el mensaje **1: main(args)** a la línea de vida de **: MainEmpleado**.

En la primera fase, el controlador envía los mensajes de instanciación parametrizados **2: «constructor»** y **5: «constructor»** hacia las líneas de vida de **empleado1 :**

**Empleado** y **empleado2 : Empleado**. Al inicializarse, cada objeto ejecuta sobre su propia línea de vida un mensaje reflexivo sincrónico (**3: validarSalario** y **6: validarSalario**), retornando el control al controlador con los mensajes 4 y 7. Posteriormente, **: MainEmpleado** interroga a ambos objetos mediante invocaciones sincrónicas sucesivas a **salarioAnual()**, capturando los valores de salario mensual por doce. Acto seguido, envía los mensajes mutadores **12: aumentarSalario()** y **13: aumentarSalario()**, concluyendo con una nueva consulta a **salarioAnual()** para desplegar en consola la comparativa salarial.

En la segunda fase, separada visualmente por una línea divisoria, el **Usuario** arranca la ejecución independiente sobre **: MainFecha** mediante el mensaje **17: main(args)**. El controlador instancia el objeto **fecha1 : Fecha** mediante **18: «constructor»(dia, mes, anio)**, solicita su visualización con **20: mostrarFecha()**, invoca sucesivamente los métodos de mutación **22: setDia(nuevoDia)**, **23: setMes(nuevoMes)** y **24: setAnio(nuevoAnio)**, y ratifica el cambio de estado con una segunda llamada a **mostrarFecha()**, tras lo cual retorna el control y culmina el hilo de ejecución.

El programa correspondiente a la **Práctica 7** resuelve el modelado de polígonos regulares e irregulares de cuatro vértices. El objetivo consiste en coordinar una jerarquía vertical de especialización geométrica, la composición inmutable de puntos en el plano y la delegación de fórmulas de cálculo de área. El sistema se conforma por la clase de soporte **Punto**, la clase base **Cuadrilatero**, la cadena de herencia descendente **Trapezoide**, **Paralelogramo**, **Rectangulo** y **Cuadrado**, y la clase controladora de prueba **MainFiguras**.

El diagrama de cardinalidad, en primer lugar, formaliza una relación de generalización lineal estricta donde **Trapezoide** hereda de **Cuadrilatero**, **Paralelogramo** hereda de **Trapezoide**, **Rectangulo** hereda de **Paralelogramo** y **Cuadrado** hereda de **Rectangulo**, trazadas mediante líneas continuas con punta triangular hueca orientada hacia la superclase inmediata.

En segundo lugar, se modela una relación de composición fuerte entre **Cuadrilatero** y la clase **Punto**, denotada por un rombo relleno en el extremo del cuadrilátero y una cardinalidad exacta de 1 a 4 acompañada del rol «contiene 4 vértices». Esta especificación define que cualquier figura cuadrilátera se constituye obligatoriamente por cuatro vértices espaciales.

En tercer lugar, el controlador **MainFiguras** presenta dependencias de instanciación con multiplicidades puntuales: una dependencia con cardinalidad de 1 a 16 hacia **Punto**, acompañada del rol «crea 16 vértices (4 por figura)», y cuatro dependencias unitarias de 1 a 1 hacia cada una de las subclases concretas de la jerarquía.

El diagrama de secuencia detalla el escenario global de instanciación de vértices, encadenamiento de constructores en cascada mediante la palabra reservada **super** y calculo de superficies geométricas. Participan en la interacción el **Usuario**, el controlador **: MainFiguras**, una línea de vida consolidada para **puntos[1..16] : Punto** y las cuatro instancias de figuras geométricas.

La interacción inicia con la ejecución del método **main**. El controlador instancia en el primer bloque cuatro coordenadas **t1..t4** e invoca el constructor de **trapezoide** mediante el mensaje **3: «constructor»(t1, t2, t3, t4)**, documentando en la barra de activación la invocación a **super()** hacia **Cuadrilatero**. Este patrón se repite en cascada sucesivamente: al construir **paralelogramo** se propaga la llamada a través de **Trapezoide** hasta **Cuadrilatero**; al construir **rectangulo** se transfiere a **Paralelogramo**; y al construir **cuadrado** se propaga hacia **Rectangulo**.

Una vez construidos los objetos, **: MainFiguras** solicita el cálculo de superficie a cada polígono mediante mensajes sincrónicos. En cada figura se ejecuta un mensaje reflexivo interno que detalla la fórmula matemática procesada. Finalmente, el controlador consolida e imprime los resultados en la salida estándar.

El propósito de la **Práctica 8** es implementar contratos de comportamiento polimórfico desacoplados mediante interfaces en Java, prescindiendo de la herencia de implementación. El programa cuantifica el impacto ambiental en términos de kilogramos de dióxido de carbono equivalente emitidos por entidades de dominios diferentes: **Edificio**, **Auto** y **Bicicleta**. El diseño integra estas clases heterogéneas bajo un contrato común para calcular sus emisiones mediante un recorrido polimórfico en la clase **Main**.

El diagrama de cardinalidad ubica en la cúspide a la interfaz **HuellaCarbono**, la cual prescribe la firma abstracta pública **+ getHuellaCarbono(): double**. Las tres clases de negocio, **Edificio**, **Auto** y **Bicicleta**, se vinculan formalmente hacia dicha interfaz mediante flechas discontinuas con punta triangular hueca, señalizando relaciones de realización de interfaz.

La clase controladora **Main** presenta dos tipos de conexiones estructurales: por una parte, dependencias de instanciación (1 a 1) hacia cada una de las clases concretas, modelando la creación directa de los objetos; por otra parte, una relación de agregación débil hacia **HuellaCarbono**, trazada con un rombo blanco en el extremo de **Main** y con multiplicidad de colección abierta de 1 a 0..* en el extremo de la interfaz. Esta asociación ostenta el rol descriptivo «almacena polimórficamente listaHuellaCarbono», expresando la capacidad del controlador para contener un conjunto de elementos compatibles con el contrato ecológico.

El diagrama de secuencia ilustra el ciclo de vida del despacho dinámico de métodos. Participan el actor **Usuario**, el controlador **: Main**, las instancias **corporativo : Edificio**, **vehiculoUrbano : Auto** y **biciRuta : Bicicleta**, y la colección genérica **listaHuellaCarbono : ArrayList<HuellaCarbono>**.

Tras el arranque del método **main**, **: Main** envía los mensajes de instanciación **«constructor»** con sus respectivos parámetros a **corporativo**, **vehiculoUrbano** y **biciRuta**. Posteriormente, inicializa la estructura dinámica mediante el mensaje **8: «constructor»()** e inserta sucesivamente las tres referencias a través de llamadas sincrónicas **add(corporativo)**, **add(vehiculoUrbano)** y **add(biciRuta)**.

A continuación, la ejecución ingresa a un fragmento combinado formal de tipo **loop**, delimitado por la condición de guarda [**para cada HuellaCarbono elemento in listaHuellaCarbono**]. Dentro del ciclo se ejecutan tres iteraciones polimórficas: en cada una de ellas, el controlador despacha el mensaje **elemento.toString()** y posteriormente invoca **elemento.getHuellaCarbono()**. Cada objeto receptor ejecuta reflexivamente su algoritmo de cálculo de emisiones y devuelve el monto. El bucle termina y **: Main** transfiere las salidas a la consola.

## 3.2 Proyecto

En la Etapa 2 del Proyecto consiste en un simulador hospitalario que administra personal clínico y pacientes, encapsulamiento de atributos inmutables, algoritmos nativos de ordenamiento y despacho polimórfico. El sistema coordina la admisión y canalización de pacientes, la asignación de consultas y prescripciones, la auditoría de guardias y la gestión disciplinaria de quejas mediante actas administrativas. El programa organiza sus componentes en los paquetes **hospital.pacientes**, **hospital.personal**, **hospital.sistema** e **interfaces**, empaquetando la interfaz clínica **AtencionMedica** en una biblioteca JAR independiente.

El diagrama de clases con cardinalidad estructura un modelo de alta cohesión. En el núcleo de la capa de servicios se ubica la interfaz **AtencionMedica**, realizada conjuntamente por las clases base abstractas **Medico** y **Enfermero**. A partir de estas, se despliegan dos árboles de generalización: **Medico** se especializa en **MedicoGeneral** y **MedicoEspecialista**, mientras que **Enfermero** se deriva en **EnfermeroEspecialista** y **JefeEnfermeria**. De manera paralela, la clase abstracta **Paciente** se ramifica en **PacienteGeneral** y **PacienteEspecialidad**, modelando adicionalmente un vínculo con la interfaz estándar **Comparable<Paciente>**.

La clase gestora **Sistema** se vincula con el resto del modelo mediante relaciones de agregación débil con multiplicidades parametrizadas con asterisco: de 1 a 0..50* hacia

**Medico**, de 1 a 0..100* hacia **Enfermero**, de 1 a 0..10* hacia **Paciente** y de 1 a 0..20* hacia **ActaAdministrativa**. El asterisco de cada cota superior muestra la capacidad de redimensionamiento dinámico de los arreglos en tiempo de ejecución.

A nivel clínico, se formalizan asociaciones que capturan las restricciones de capacidad operativa: la asociación entre **Medico** y **Paciente** delimita dos roles diferenciados, una multiplicidad de 1 a 0..1 para el rol **pacienteEnConsulta** y de 1 a 0..10 para el arreglo **pacientesAsignados**. A su vez, la asociación entre **Enfermero** y **Paciente** fija una multiplicidad de 1 a 0..3 bajo el rol **pacientesAsignados**. La clase **JefeEnfermeria** presenta una asociación de supervisión con multiplicidad de 1 hacia 0..* respecto a **Enfermero**, mientras que la entidad **ActaAdministrativa** establece asociaciones unitarias estrictas de 1 a 1 hacia el **Paciente** denunciante y el **Enfermero** reportado.

El diagrama de secuencia modela el «Ciclo Clínico Completo», articulando la precarga del sistema, la canalización inteligente, la consulta médica y la verificación de polimorfismo. Participan en el escenario el actor externo **Personal Admin**, la clase de control **: Main**, la instancia coordinadora **hospital : Sistema** y un conjunto de objetos de dominio: **drSalas : MedicoGeneral**, **draClara : MedicoEspecialista**, **licAna : EnfermeroEspecialista**, **jefaSofia : JefeEnfermeria**, **juanPerez : PacienteGeneral** y **mariaGarcia : PacienteEspecialidad**.

El escenario inicia con la instanciación de **hospital** y la precarga de los profesionales clínicos y pacientes mediante constructores parametrizados. A continuación, **juanPerez** solicita atención médica enviando el mensaje **10: solicitarConsulta(hospital)**. El sistema ejecuta una auto-evaluación reflexiva mediante **11: asignarPacienteAutomatico** y, reconociendo su condición de consulta básica, deriva la custodia al médico general mediante **12: admitirPaciente(juanPerez)**. En paralelo, **mariaGarcia** envía **14: solicitarConsulta(hospital)**; el sistema evalúa su condición de alta especialidad cardiológica y triaje de urgencia nivel 4, admitiéndola en el consultorio de **draClara**.

Posteriormente, se modela la fase de consulta y tratamiento evidenciando sobrecarga de métodos: **drSalas** atiende a Juan invocando **18: darConsulta** y prescribe el tratamiento mediante **19: darTratamiento(juanPerez)** empleando la firma de un argumento; por su parte, **draClara** consulta a María y prescribe **21: darTratamiento(mariaGarcia, 'Enalapril 10mg c/12h')** haciendo uso de la variante sobrecargada de dos argumentos. Acto seguido, la jefa de enfermería ejecuta **23: supervisarGuardia** sobre la plantilla. El diagrama incorpora hacia el cierre un fragmento combinado **loop** donde se despacha de manera polimórfica la operación **staff.darConsulta(paciente)** a través de referencias de la interfaz **AtencionMedica**, culminando con el reporte de estadísticas globales emitido por **hospital**.

# 4. Resultados

## 4.1 Resultados de la Práctica 9

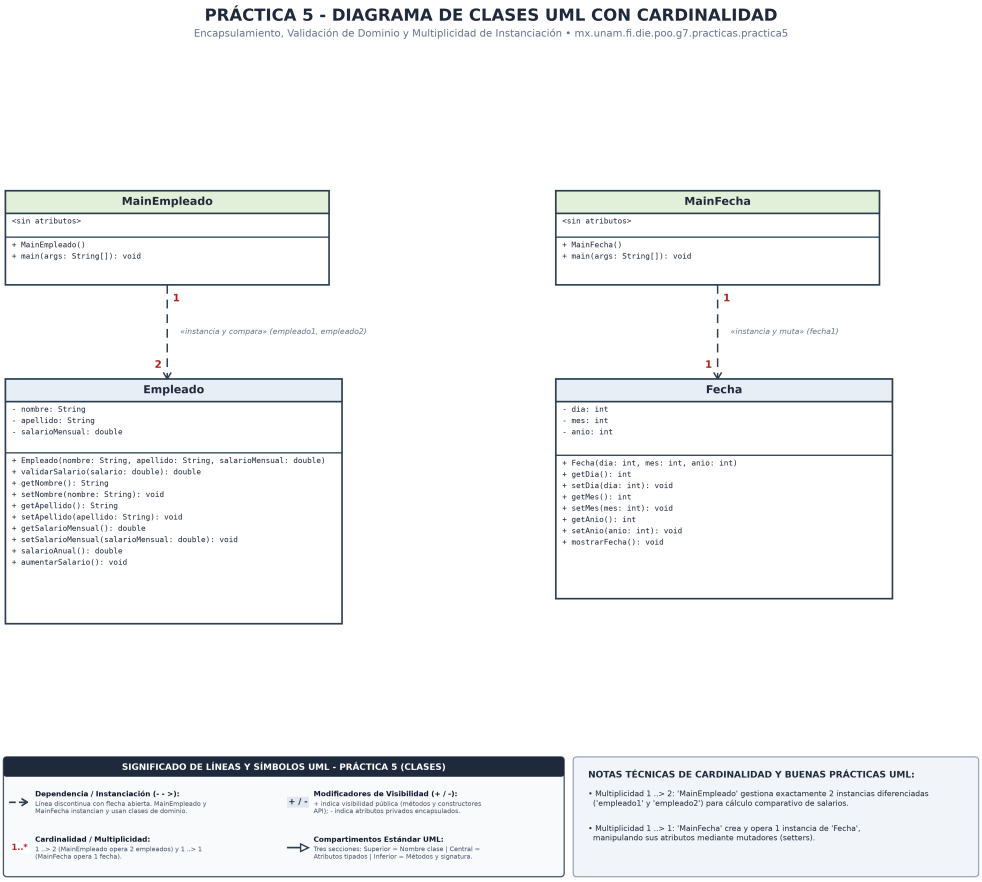



Figura 1: Diagrama de clases UML con cardinalidad correspondiente a la Práctica 5.

Ver imagen 01_P5_Clases_Cardinalidad.png en GitHub

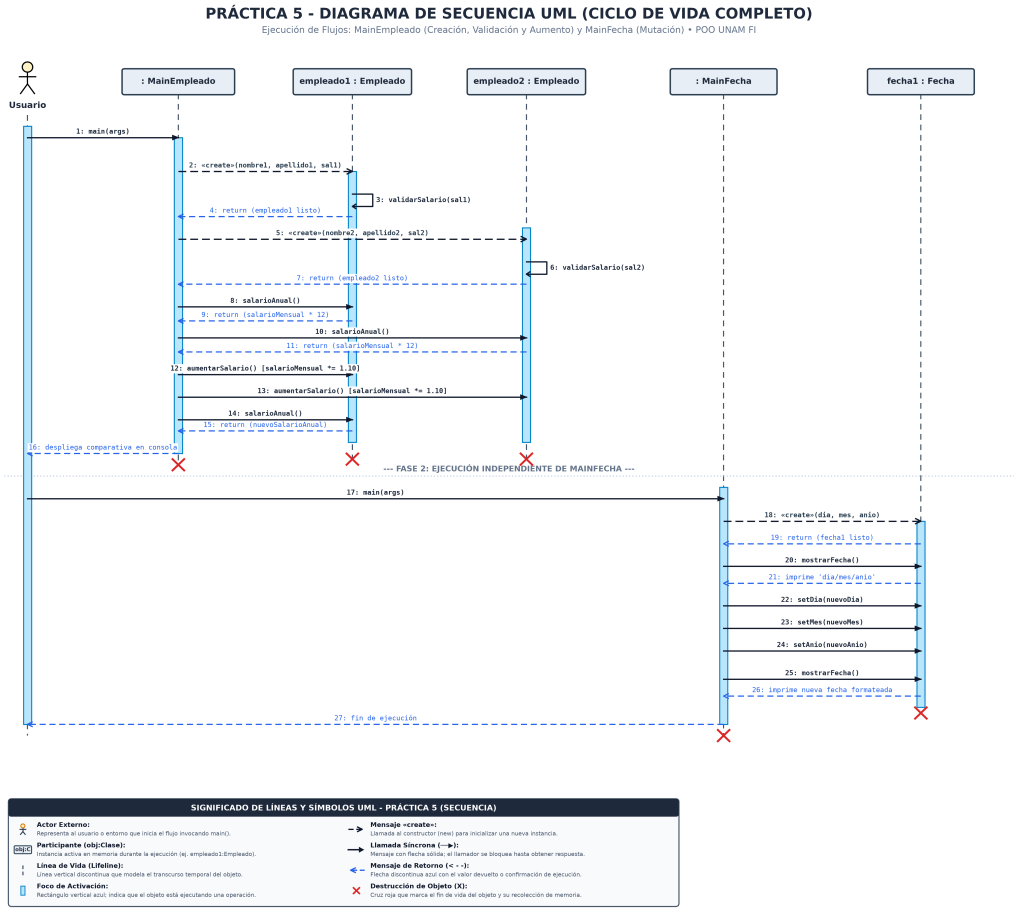



Figura 2: Diagrama de secuencia UML del ciclo de vida completo correspondiente a la Práctica 5.

Ver imagen 02_P5_Secuencia.png en GitHub

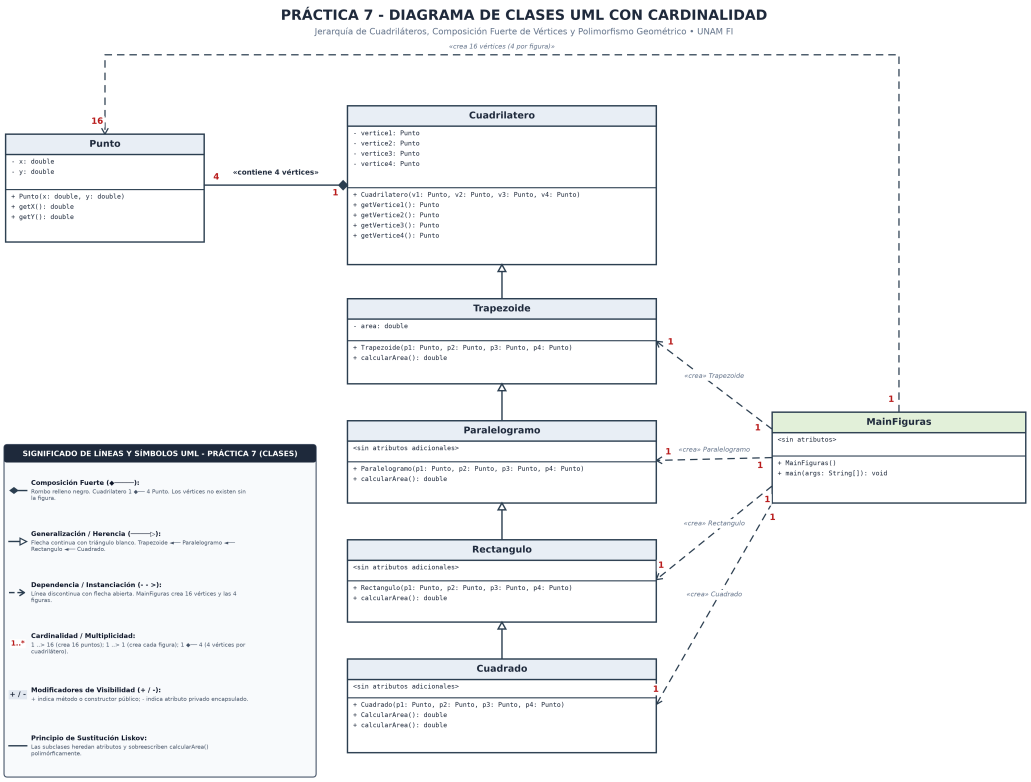



Figura 3: Diagrama de clases UML con cardinalidad correspondiente a la Práctica 7.

Ver imagen 03_P7_Clases_Cardinalidad.png en GitHub

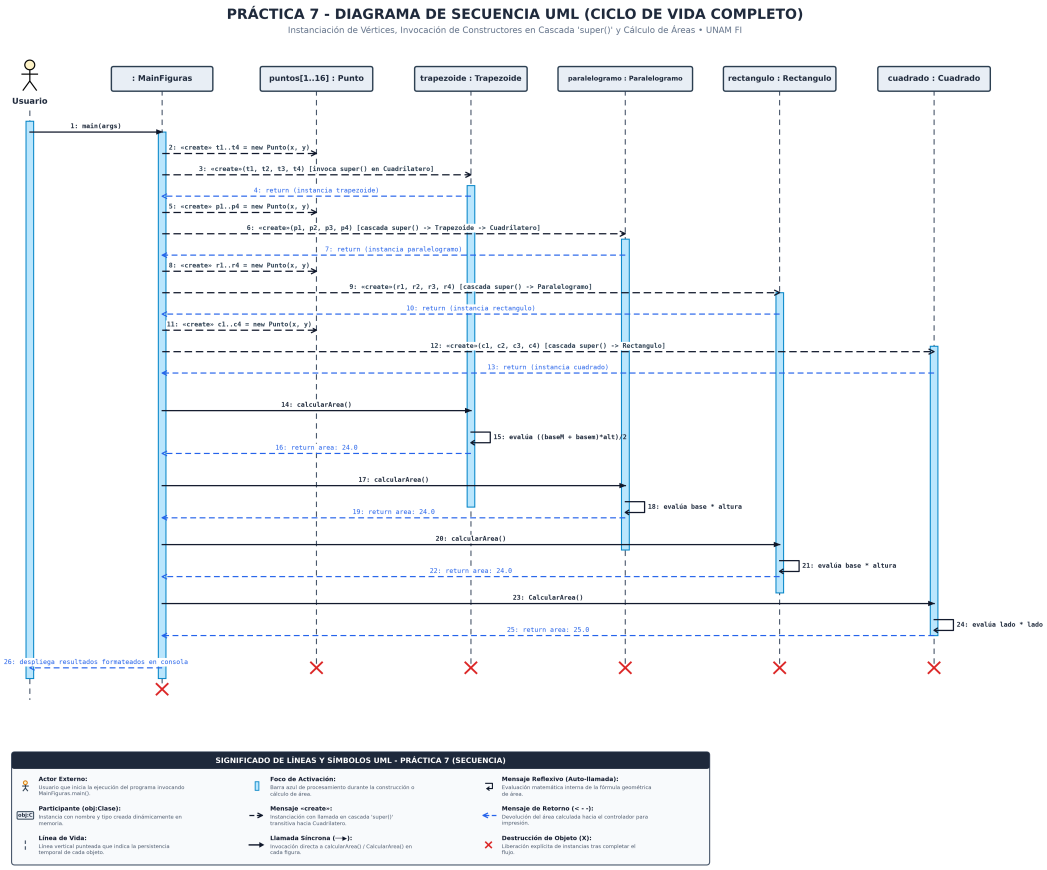



Figura 4: Diagrama de secuencia UML del ciclo de vida completo correspondiente a la Práctica 7.

Ver imagen 04_P7_Secuencia.png en GitHub

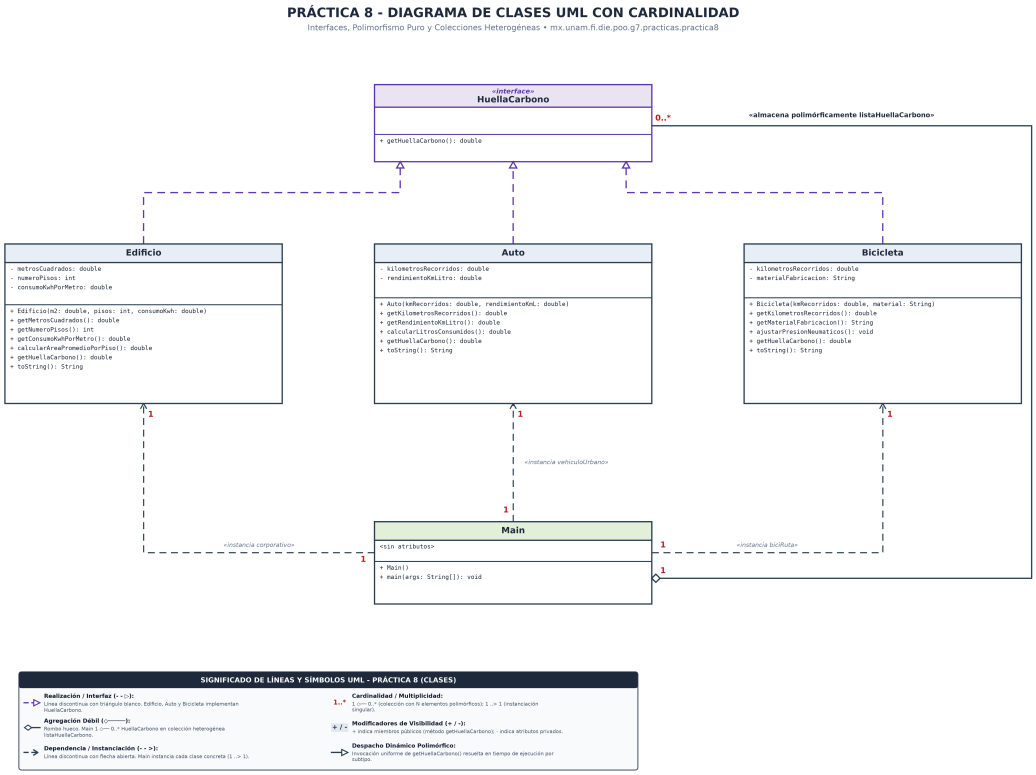



Figura 5: Diagrama de clases UML con cardinalidad correspondiente a la Práctica 8.

Ver imagen 05_P8_Clases_Cardinalidad.png en GitHub

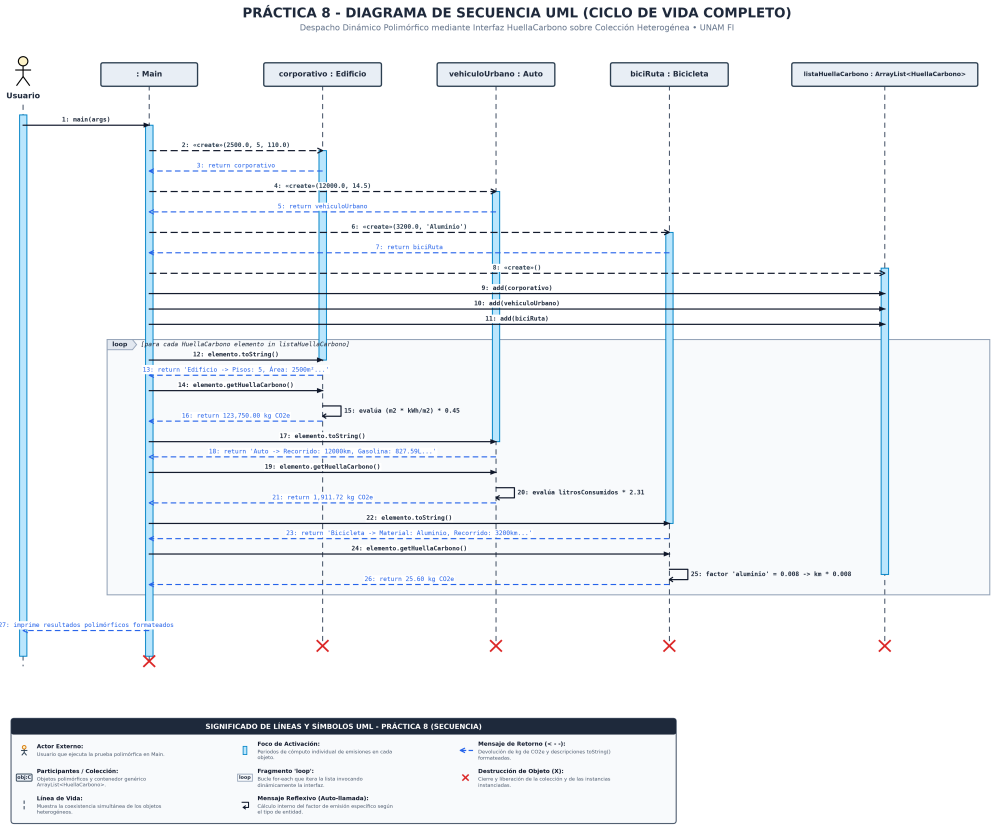



Figura 6: Diagrama de secuencia UML correspondiente a la Práctica 8.

Ver imagen 06_P8_Secuencia.png en GitHub

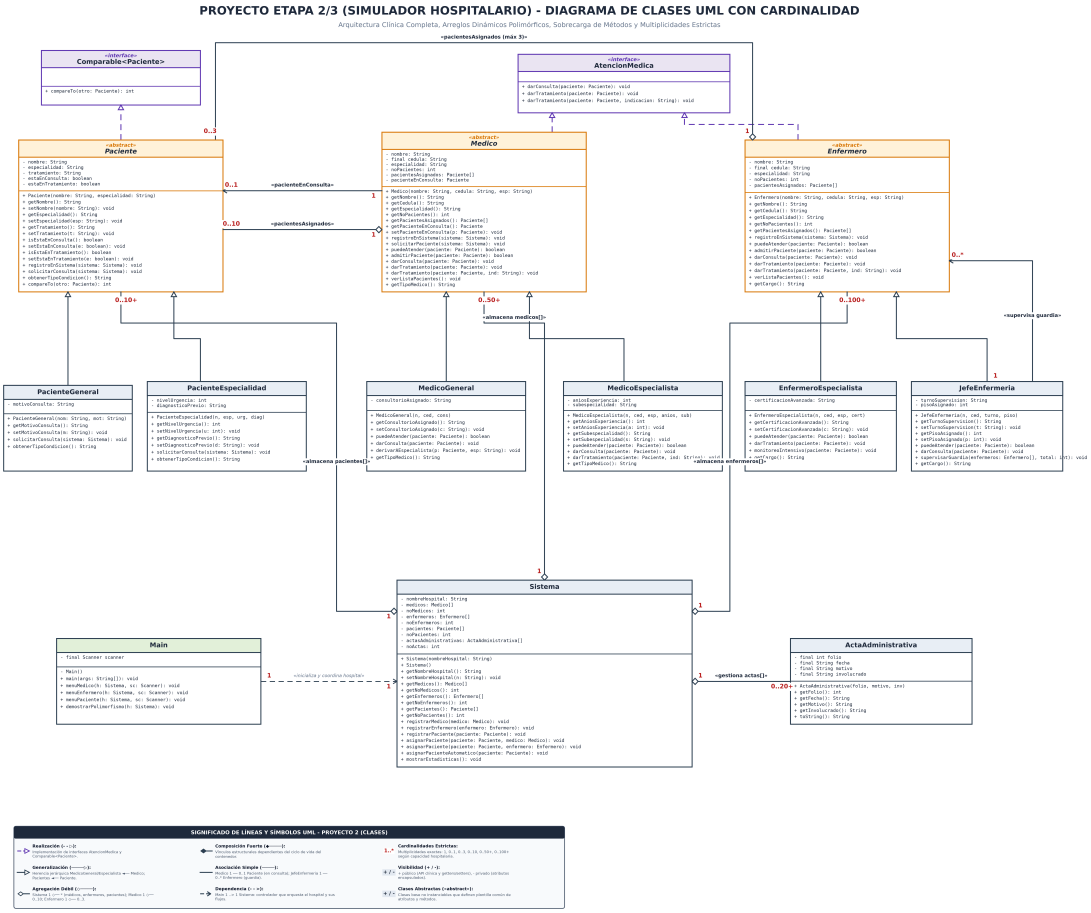



Figura 7: Diagrama de clases UML con cardinalidad correspondiente a la Etapa 2 del Proyecto.

[Ver imagen 07_Proyecto2_Clases_Cardinalidad.png en GitHub]

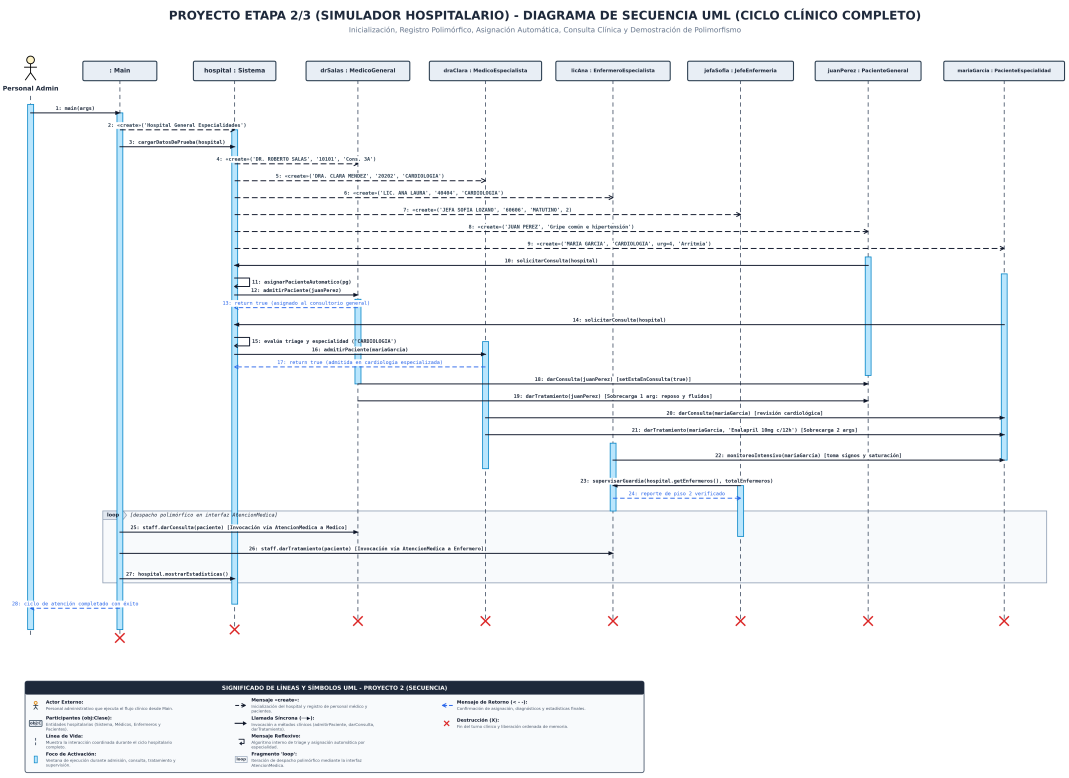



Figura 8: Diagrama de secuencia UML del ciclo clínico completo correspondiente a la Etapa 2 del Proyecto.

Ver imagen 08_Proyecto2_Secuencia.png en GitHub

# 5. Conclusiones

Los resultados obtenidos del análisis evidencian que los diagramas de cardinalidad y de secuencia proporcionan una comprensión del software que va más allá de una simple inspección del código fuente. El contraste entre las representaciones visuales y los programas desarrollados demostró que la cardinalidad es la representación gráfica de las reglas de negocio. Asimismo, los diagramas de secuencia confirmaron la importancia del modelado temporal para documentar el comportamiento polimórfico en tiempo de ejecución. El uso equilibrado entre modelos de clases con cardinalidad y diagramas de secuencia consolida un marco documental.

# 6. Referencias

[1] Object Management Group, "OMG Unified Modeling Language (OMG UML), Superstructure, Version 2.4.1", formal/2011-08-05, Ago. 2011.

[2] M. Fowler, *UML Distilled: A Brief Guide to the Standard Object Modeling Language*, 3a ed. Boston, MA, EE. UU.: Addison-Wesley, 2004.

[3] G. Booch, J. Rumbaugh e I. Jacobson, *The Unified Modeling Language User Guide*, 2a ed. Upper Saddle River, NJ, EE. UU.: Addison-Wesley, 2005.